VLSI Testing: Homework 1
Student reference report

Quartz-4b7e

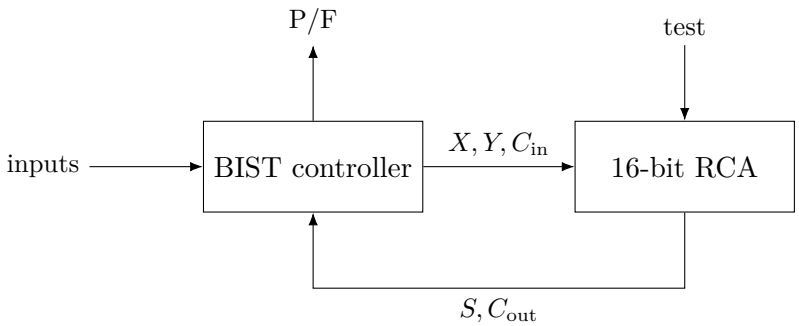
Status. This is an editable English reconstruction of the anonymized student report. It preserves the student's methods and reported conclusions. Independent verification notes appear at the end and distinguish confirmed results from errors. The original statement and anonymized report are included alongside this document.

1 BIST for a 16-bit ripple carry adder

The design consists of a full adder, a 16-stage ripple carry adder (RCA), a BIST controller, and a top-level module. Each full adder uses

$$w_1 = X_i \oplus Y_i, \quad S_i = w_1 \oplus C_i, \quad C_{i+1} = X_i Y_i + w_1 C_i.$$

The RCA connects each carry to the next stage. In normal mode the BIST controller registers the external operands and carry input. In test mode it supplies eight fixed patterns. The top-level module connects the controller's operand outputs to the RCA and returns the RCA's result to the controller.



1.1 Fault injection

In the retained reference source, the first full adder's carry is forced to one whenever `test=1`. The injected signal is `carry[0]`, after `fa0`. A source comment names another cell, but the wiring and the report's description identify the first cell. This intentionally faulty variant is retained as submitted.

1.2 Pattern generator and response comparison

The pattern counter is reset to zero, advances during test mode, and stops at nine. The controller applies the following patterns at counter values zero through seven and compares the previous cycle's result at counter values one through eight. Here `FFFF`, `5555`, and `AAAA` are hexadecimal values.

Pattern index	X	Y	C_{in}	Expected $\{C_{\text{out}}, S\}$
0	0000	0000	0	0:0000
1	0000	FFFF	0	0:FFFF
2	0000	FFFF	1	1:0000
3	FFFF	0000	0	0:FFFF
4	FFFF	0000	1	1:0000
5	FFFF	FFFF	1	1:FFFF
6	5555	5555	1	0:AAAB
7	AAAA	AAAA	0	1:5554

The student report uses `pf=1` for a mismatch and zero for a match. In normal mode and during reset it assigns high impedance to `pf`. The comparisons are individual cycle results rather than a retained cumulative fault flag.

1.3 Testbench and reported waveform interpretation

The supplied testbench runs normal mode from 0 to 100 ns and then enables test mode. Its normal arithmetic examples are:

X	Y	C_{in}	S
300	59	0	359
15	20	1	36
1300	56	1	1357
50	100	0	150

The original prose accidentally repeats 50 for the second operand in the last example; the supplied testbench uses 100. The report shows the fault injection producing result 2 for $0 + 0 + 0$ and result 1 with carry one for $0 + \texttt{FFFF} + 0$. Both cause a mismatch in the following clock cycle. The input $0 + \texttt{FFFF} + 1$ produces the correct result despite this fault, so the comparison returns zero. The report also observes a correct result for `5555`+`5555`+1. These examples illustrate that a fault can be masked under some patterns.

2 Fault equivalence and dominance collapsing

For the second question the student draws equivalence connections in green, dominance arrows in orange, and selected faults in red. The reported retained fault list is

$$\{A/1, B/1, C/0, D/0, E/1, G/1, G/0, H/0, I/0\}.$$

The circuit labels can be represented by the following equations, keeping the fanout stem F and its branches G, I distinct:

$$H = AB, \quad F = C + D, \quad G = F, \quad I = F, \quad V = \overline{G}E, \quad Z = H + I + V.$$

The annotated original PDF preserves the student's full fault diagram. This list is recorded as a student result rather than a new proof of a minimal fault set.

3 An internal fault missed by the five vectors

The third circuit implements the full-adder sum and carry functions

$$w_1 = B \oplus C, \quad w_2 = Aw_1, \quad w_3 = BC, \quad \text{out1} = A \oplus w_1, \quad \text{out2} = w_2 + w_3.$$

A	B	C	w_1	w_2	w_3	out1	out2
0	0	0	0	0	0	0	0
1	0	0	0	0	0	1	0
0	1	0	1	0	0	1	0
1	1	1	0	0	1	1	1
0	0	1	1	0	0	1	0

The supplied testbench applies these five vectors. Since w_2 remains zero under every vector, w_2 stuck at zero cannot be activated and is not detected. This is the report's valid example of an internal undetected fault. The report additionally claims that $A/0$ and $w_1/0$ are undetected because of dominance; those additional claims fail when both output ports are observed.

4 Independent verification notes

The original RTL and testbenches remain unchanged. The BIST counter advances even when `En=0`, while pattern outputs hold, so pausing enable does not pause the test sequence. The P/F result is overwritten by each comparison and defaults to zero after the checking states; it is not a sticky final failure indication. The eight patterns present only six distinct input triples to the first full adder, missing 001 and 110. A cell fault that changes an output only on either missing triple can escape these tests. Therefore the implementation does not establish complete coverage for arbitrary single-cell faults.

For question 3, $A/0$ is detected by input 100 at `out1`, and $w_1/0$ is detected by input 001 (also 010) at `out1`. These are counterexamples to the report's additional claims. The statement's five-vector set and the submitted testbench have the same vectors in a different order.

5 Retained source files

The `source` directory contains the BIST controller, full adder, RCA, top level, its testbench, and the separate question-3 circuit and testbench. The two testbenches both declare module `tb`; compile them as separate simulations. Their `$fsdbDumpfile` and `$fsdbDumpvars` calls require an FSDB-capable simulator or PLI. Portable checks supplied separately do not alter these originals.